Ders 27 Genişletilmiş Notu: Vize Soru Çözümleri, Böl ve Fethet ile En Küçüğü Bulma, Not Dağılımı, Dönem Projesi ve Final Bilgilendirmesi

Ders 27 Genişletilmiş Notu: Vize Soru Çözümleri, Böl ve Fethet ile En Küçüğü Bulma, Not Dağılımı, Dönem Projesi ve Final Bilgilendirmesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Vize Test Sorularının Çözümü
- 2. Büyüme Hızlarının Karşılaştırılması
- 3. Rekürans Bağıntılarının Çözümü
- 4. Hash Yöntemiyle İki Kümenin Kesişimi
- 5. Böl ve Fethet ile En Küçüğü Bulma
- 6. Dönem Projesinin Teknik Konusu: İşbirlikçi Filtreleme ile Kitap Önerisi
- Örnek Sorular
- Dersin Toparlaması

Bu kayıt dönemin son dersidir. Yeni konu anlatılmadı. Hoca vize sorularını tek tek çözdü, özellikle sınıfın büyük kısmının yanlış yoldan çözdüğü dördüncü klasik soruyu ayrıntılı anlattı. Ardından dönem projesinin teknik içeriği üzerinden geçildi. Not dağılımı, sınav düzeni ve ödev kontrolü gibi idari kısımlar bu nota alınmadı.

1. Vize Test Sorularının Çözümü

Hoca test bölümünde büyük bir sorun olmadığını, ama soru işareti kalmaması için hepsinin üzerinden hızlıca geçeceğini söyledi.

1.1 Döngüde Toplama Sayısı

İlk soru, bir kod parçasında kaç toplama işlemi yapıldığını soruyordu. Döngü değişkeni her adımda iki katına çıkıyordu ($k = k \cdot 2$ biçiminde). Böyle bir döngü $\log n$ kez döner. Hocanın verdiği sezgi: 8'e kadar gidiyorsanız işlem 3 kez, 16'ya kadar gidiyorsanız 4 kez yapılır. Yani karmaşıklık $\log n$ mertebesindedir.

1.2 Linear Probing'de Silinen Gözler

Soru şuydu: Çakışma probleminin doğrusal yoklama (linear probing) ile çözüldüğü bir hash tablosunda boş gözler, boşalma biçimine göre "empty" (hiç dolmamış) ya da "deleted" (silinmiş) bilgisini taşır. Aşağıdaki iki varsayımın doğruluğunu değerlendirin.

- **"i. adres deleted, $i+1$. adres empty ise i. adres empty olarak işaretlenebilir."** Doğru. Bir arama i'ye gelip devam etseydi zaten $i+1$'deki boş gözde duracaktı. Deleted gözün arkasında aramayı sürdürmeyi gerektiren bir eleman yoktur. Bu yüzden deleted gözü empty yapmanın bir sakıncası yoktur.

- **"i. adres deleted, $i+1$. adreste doğru adrese yerleşmiş bir eleman varsa i. adres empty olarak işaretlenebilir."** Yanlış. $i+1$ boş değilse ve i deleted ise, aradan bir eleman silinmiş demektir. i empty yapılırsa, i'den önceki bir adrese hash'lenip i'nin ötesine yerleşmiş elemanlar için yapılan arama i'de "boş göz bulundu" diye yarıda kesilir. Bu yüzden i. göz deleted olarak kalmalıdır.

  Ders sonrası not: Altyazıda hoca ikinci şık için "buranın direkt olarak kalması gerekiyor, empty olarak düzeltilmemiz gerekiyor" diyor. Anlatımın bütününden kastedilen, gözün deleted olarak kalması ve empty yapılmamasıdır.

1.3 $2n^3 - 7n + 1$ İçin Hangi Gösterim Kullanılamaz?

Şıklar $O(n^3)$, $\Theta(n^3)$, $O(n^4)$ ve $\Omega(n)$ ile "hepsi kullanılabilir" seçeneğiydi. Hoca tek tek baktı:

- $O(n^3)$ ve $\Theta(n^3)$ doğrudur, en büyük terim n^3'tür.
- $O(n^4)$ de doğrudur. Genelde kullanmasak da daha yüksek bir üst sınır (upper bound) gösterdiği için algoritmanın karmaşıklığı $O(n^4)$ olarak da ifade edilebilir.
- $\Omega(n)$ de doğrudur. n, n^3'ten daha düşük olduğu için alt sınır (lower bound) olarak kullanılabilir.

Bu yüzden doğru cevap "hepsi kullanılabilir" seçeneğiydi.

1.4 Hash Tablosuna Yerleştirme Sırası

Soru: Sayılar $h(x) = x \bmod 10$ fonksiyonuyla bir hash tablosuna yerleştiriliyor. 6 adım sonra tablo aşağıdaki gibi olduğuna göre sayılar hangi sırayla yerleştirilmiştir?

Adres	2	3	4	5	6	7
Değer	42	23	34	52	46	33

Hoca, farklı çözüm yolları olsa da şıklar üzerinden gidilebileceğini söyledi. İlk iki şıkta bir noktada çelişki çıkıyor. Örneğin bir şıkta 52, 23'ten önce geliyordu. O durumda 52, 2 dolu olduğu için 3'e yerleşirdi. Ama tabloda 3'te 23 var, demek ki bu sıra doğru değil. Diğer şıkta da 33'ün yerleşmesi gereken yer tabloyla uyuşmuyordu.

Doğru şık 46, 34, 42, 23, 52, 33 sırasıdır. Adım adım:

- 46 → $46 \bmod 10 = 6$, boş, 6'ya yerleşir.
- 34 → 4, boş, 4'e yerleşir.
- 42 → 2, boş, 2'ye yerleşir.
- 23 → 3, boş, 3'e yerleşir.
- 52 → 2 dolu (42), 3 dolu (23), 4 dolu (34). İlk boş göz olan 5'e yerleşir.
- 33 → 3 dolu, 4, 5 ve 6 da dolu. İlk boş göz olan 7'ye yerleşir.

Hoca D ve E şıkları kontrol edildiğinde onların da hatalı çıkacağını söyledi.

1.5 Algoritma Karşılaştırma Ölçütleri

Soru: Aşağıdakilerden kaç tanesi iki algoritmanın verimlilik yönünden karşılaştırılmasında kullanılan ölçütler arasında yer alabilir? Hoca bunun muhtemelen en çok hata yapılan soru olduğunu tahmin etti ve cevabı sınıfa sordu.

Soru: Hangileri doğru? **Cevap (bir öğrenci):** Yer karmaşıklığı, zaman karmaşıklığı, çalışma süresi ve büyüme derecesi. Hoca onayladı: Cevap 4'tür. Verilen girdi (input) sayısının iki algoritmanın karşılaştırılmasıyla bir ilgisi yoktur. Kod uzunluğunun da verimlilik karşılaştırmasıyla bir ilgisi yoktur.

1.6 İkili Ağaca Ekleme, En Kötü Durum

Soru: n elemanlı ikili ağaca eleman yerleştirme işleminin en kötü durumdaki karmaşıklığı nedir? Cevap $O(n)$'dir. Ağacın bir tarafa yığılma ihtimali göz önüne alınır: Elemanlar sıralı gelirse ağaç tek bir zincire dönüşür ve yeni eleman en dipteki yere kadar bütün düğümlerden geçerek yerleşir. Bu sorunun cevabı sistemde yanlış şık olarak girilmişti, hoca bunu düzeltti.

1.7 Sıralama Algoritmalarını Ara Adımlardan Tanıma

Bu soruda aynı dizinin beş farklı sıralama algoritmasıyla sıralanırken bir ara adımdaki hâlleri sütunlar hâlinde verilmişti. Birinci sütun sırasız diziyi, yedinci sütun sıralı diziyi gösteriyordu. Aradaki beş sütun hangi algoritmaya ait olduğuna göre eşleştirilecekti. Doğru sıra: seçmeli sıralama (selection sort), hızlı sıralama (quick sort), birleştirmeli sıralama (merge sort), eklemeli sıralama (insertion sort) ve yığın sıralaması (heap sort).

Bir öğrenci, dizilerin dikey değil yatay verilmesinin ya da daha az sayıyla verilmesinin okumayı kolaylaştıracağını söylemişti. Hoca bu eleştiriyi değerlendirilebilir buldu. Ardından çözüm yolunu anlattı. En kolay tanınan merge sort, sonra insertion sort'tur. Bu ikisi bulununca geriye kalanlar çok daha kolay ayırt edilir.

- **Merge sort:** Ortadaki sütuna bakınca dizinin belli bir noktaya kadar kendi içinde sıralı, o noktadan sonra da kendi içinde sıralı olduğu görülür. Bu tek bakışta, ikili birleştirmeler sonucu bu noktaya gelinmiş olabileceğini, yani merge sort olduğunu gösterir.
- **Insertion sort:** Dizinin baş kısmı her adımda sıralı ilerlemiştir. Bu sütunda 86, 57 ve 83 henüz yerlerine yerleşmemiştir. Baştaki kısım sıralı olduğu ve geride kalan elemanlar başlangıçtaki sıralarını koruduğu için bu bir insertion sort'tur.
- **Selection sort:** Baş kısım 11, 18, 22, 28, 30, 37, 39, 50, 56, 57, 58, 60 diye küçükten büyüğe gidiyor. Ama bundan sonra 92, 66, 78 geliyor ve kalan elemanlar başlangıçtaki sırada değil. Insertion sort ile temel fark budur. Insertion sort'ta öteki elemanlar sırayı kaybetmeden arka planda dizili kalır, sırası gelen eleman olması gereken yere yerleşir. Selection sort'ta ise en küçük eleman öne alınırken yerindeki eleman geriye taşınır, yani yer değiştirmeler (swap) kalan kısmın sırasını bozar.

Hoca heap sort'u anlamanın biraz daha zor olabileceğini söyleyerek sona bıraktı ve onun ayrıntısına girmedi. Bu sorunun en fazla süre harcanacak soru olduğunu belirtti.

> **Ders sonrası not:** Hoca doğru sırayı söylerken ve sonda selection sort'tan söz ederken "sequential sort" dedi. Doğru ad seçmeli sıralamadır (selection sort).

2. Büyüme Hızlarının Karşılaştırılması

Klasik bölümün ilk sorusunda fonksiyon çiftlerinin büyüme dereceleri karşılaştırılacak ve her satıra yalnız O, Θ ya da Ω'dan biri yazılacaktı. Soruda "$f(n) = \Omega(g(n))$ ise birinci sütuna Ω yazınız" diye bir örnek verilmişti. Hoca bunun dışında biçimlerde cevap verenlerin, ya da cevabı anlaşılamayacak şekilde yorumlayanların doğru kabul edilmediğini, açık biçimde $f(n) = O(g(n))$ diye yazanların ise doğru sayıldığını söyledi.

- n^2 **ile** n^3**:** n^3 daha büyük olduğu için $n^2 = O(n^3)$, cevap O.
- **İkinci satır:** Cevap Ω. Hoca, karşılaştırılan fonksiyonun $n \log n$'den küçük olduğunu söyleyerek Ω yazılması gerektiğini belirtti.
- 1 **ile** $2 + \sin n$**:** Hoca bu satırda çok hata yapıldığını söyleyip cevabı sınıfa sordu. Cevap gelmeyince kendisi açıkladı: n ne olursa olsun $\sin n$ belli bir aralıkta kalır. İki sabitin toplamı da sabittir, yani $2+\sin n$ değeri n'e bağlı olarak büyümez. İki taraf da sabit olduğu için birbirine eşdeğerdir ve cevap Θ'dır. Bir öğrenci sohbet kısmına da "Θ olacak" diye yazmıştı.
- $n \log n$ **ile** $n^{150/100}$**:** $n^{150/100} = n^{1.5} = n \cdot n^{0.5}$. Burada n'in üssü olan çarpan, doğal olarak $\log n$'den daha büyük bir çarpandır. Bu yüzden $n \log n = O(n^{150/100})$, cevap O.
- $\log(\sqrt{10}\, n)$ **ile** $\log(n^3)$**:** $\log(n^3) = 3 \log n$ olur ve 3 bir sabit olduğu için atılabilir. Diğer tarafta $\log(\sqrt{10}\, n) = \log \sqrt{10} + \log n$ olur, $\sqrt{10}$ da sabit olduğu için atılabilir. İki tarafta da $\log n$ kalır, cevap Θ.

- $n!$ **ile** $(n+1)!$**:** Hoca bunun en çok yanlış yapılan satır olduğunu söyledi. $(n+1)! = (n+1) \cdot n!$ olduğundan $n!$'ler sadeleştirilince karşılaştırma 1 ile $n+1$ arasında kalır. $n+1$, 1 sabitinden büyük olduğu için cevap O'dur.

  Ders sonrası not: Son satırda hoca "$n-1$, 1 sabitinden daha büyük" dedi; sadeleştirme sonrası kalan ifade $n+1$'dir. Asıl önemli nokta $(n+1)!/n! = n+1$ oranının sınırsız büyümesidir; bu yüzden $n! = O((n+1)!)$ olur ama Θ olmaz. Üçüncü satırda da $-1 \leq \sin n \leq 1$ olduğundan $1 \leq 2 + \sin n \leq 3$ olur; bu sınırlar Θ cevabını kesinleştirir.

3. Rekürans Bağıntılarının Çözümü

Soru, rekürans bağıntılarının adımlar gösterilerek çözülmesini istiyordu. Hoca çözümü ekranda yazarak yaptı.

3.1 $x_n = x_{n-1} + 5$, $x_1 = 0$

Bağıntı $n > 1$ için $x_n = x_{n-1} + 5$, başlangıç koşulu $x_1 = 0$ olarak verilmişti. Burada geriye doğru yerine koyma (backward substitution) uygulanması bekleniyordu.

$$x_n = x_{n-1} + 5$$

x_{n-1} yerine $x_{n-2} + 5$ yazılır:

$$x_n = (x_{n-2} + 5) + 5 = x_{n-2} + 5 \cdot 2$$

Bir kez daha geriye dönülür, x_{n-2} yerine $x_{n-3} + 5$ yazılır:

$$x_n = x_{n-3} + 5 \cdot 3$$

Bu şekilde devam edilince genelleştirme yapılabilir:

$$x_n = x_{n-i} + 5i$$

Başlangıç koşulu x_1'e ulaşmak için $i = n-1$ alınır:

$$x_n = x_1 + 5(n-1) = 0 + 5n - 5 = 5n - 5$$

3.2 $T(n) = 4T(n/2) + n^2$, $T(1) = 1$

Hoca bunun görece kolay bir soru olduğunu söyledi. Hem master teoremi hem de geriye doğru yerine koyma kullanılabilir. Master teoremiyle:

- $a = 4$: Özyinelemeli çağrının katsayısı.
- $b = 2$: Problem 2'ye bölünüyor.
- $d = 2$: Ek iş n^2 olduğu için.

$b^d = 2^2 = 4 = a$ olduğundan master teoreminin $a = b^d$ durumu geçerlidir. Hoca bu durumun formülünü sınıfa sordu, gelen cevaplar net olmayınca doğrudan sonucu yazdı:

$$T(n) \in \Theta(n^2 \log n)$$

Soru: Soruda gerçekten n^2 mi yazıyordu? Ben $2n$ olarak okuyup çözdüm. **Cevap:** Hoca soruyu ekranda açıp gösterdi: n^2 yazıyordu. Üs işaretinin görünmemesinden kaynaklanan bir yazım hatası olabileceğini, ama sınıfın büyük çoğunluğunun ifadeyi n^2 olarak aldığını söyledi. $2n$ olarak doğru çözenlerin durumunu ayrıca kontrol edeceğini belirtti.

Ek bilgi (derste anlatılmadı): Master teoremi, $T(n) = a\,T(n/b) + f(n)$ ve $f(n) \in \Theta(n^d)$ ($d \geq 0$) için üç durum verir. $a < b^d$ ise $T(n) \in \Theta(n^d)$; $a = b^d$ ise $T(n) \in \Theta(n^d \log n)$; $a > b^d$ ise $T(n) \in \Theta(n^{\log_b a})$. Bu soruda ikinci durum geçerlidir.

Ders sonrası not: Hoca $a = b^d$ durumunun formülünü derste hatırlatamadı. Doğrusu $\Theta(n^d \log n)$'dir; $d = 2$ için $\Theta(n^2 \log n)$ çıkar. $n = 2^k$ için geriye doğru yerine koyma $T(n) = n^2(\log_2 n + 1)$ verir; bu tam çözüm python ile $k = 0, \ldots, 14$ için doğrulandı ve $\Theta(n^2 \log n)$ ile uyumludur.

4. Hash Yöntemiyle İki Kümenin Kesişimi

Üçüncü klasik soru: İki ayrı dizide verilen iki kümenin kesişim kümesini hash yöntemiyle bulan algoritmanın standart C dilinde tam programını yazın. Hash tablosunda çakışma problemini linear probing yaklaşımıyla çözün.

Hoca bu soru derste hash yöntemleri ve linear probing'in algoritmik olarak gösterildiği bir konu olduğu için kod üzerinden uzun uzun gitmedi. Bunun yerine çözümün adımlarını ve en sık yapılan hataları anlattı.

Birinci dizinin yerleştirilmesi. Birinci dizinin elemanları hash'lenerek tabloya yerleştirilir. Dikkat edilecek noktalar:

- Hash tablosu için bir boyut seçilmeli ve bu boyutun asal sayı olması önemlidir.
- Hash fonksiyonu kullanılmalıdır. Hash fonksiyonu kullanmadan yapanlardan puan kırıldı. Soruda hash işlemleri için hazır fonksiyon kullanılmaması da ayrıca not edilmişti.
- Bir elemanı yerleştirirken göz boş (null) olmadığı sürece adres her seferinde bir artırılır. Bunu özyinelemeli (recursive) ya da yinelemeli (iterative) yapmak, ya da fonksiyonlara bölmek sorun değildir.
- İlk yerleştirmede tablonun dolu olup olmadığı kontrol edilmelidir. Bu, en sık yapılan hatalardan biriydi.

İkinci dizinin aranması. Yerleştirmeden sonra ikinci dizinin her elemanı aynı mantıkla aranır: Eleman hash fonksiyonundan geçirilir, üretilen adrese bakılır. Linear probing kullanıldığı için boş göz görülene kadar ilerlenir ve her gözde elemanın aranan değere eşit olup olmadığına bakılır. Eşit değilse devam edilir. Bütün tabloyu dolaşma ihtimali de hesaba katılmalıdır. Tablo doluysa ve eleman hiç bulunamadıysa, döngüden çıktıktan sonra hangi sebeple çıkıldığı mutlaka kontrol edilmelidir.

Hocanın karşılaştığı hatalar şunlardı:

- Döngüden çıktıktan sonra çıkış sebebinin kontrol edilmemesi.
- Aramanın boş göz görülene kadar sürdürülmemesi.
- Boş göze kadar ilerlenip elemanın aranan değere eşit olup olmadığına bakılmaması.
- Bağlı listeyle çözülmesi. Soruda ayrı zincirleme (separate chaining) istenmemişti, bu çözümler düşük puan aldı.

Hoca bu sorunun sınavın en iyi çözülen sorusu olduğunu söyledi. Bu soruyla bağlantılı olarak kod yazımı hakkında genel bir vurgu yaptı:

> **Hocanın vurgusu:** Sınavda C kodu istenir ve kodun çalışması çok iyi olur. Ama sınav süresinde, finalde de geçerli olmak üzere, yazdığınız kodun algoritmik kısmının doğru olması, kodun söz dizimi (syntax) hatalarından ya da ufak tefek hatalardan dolayı çalışmamasından daha önemlidir. Odaklanmanız gereken yer algoritmik kısımdır. Küçük söz dizimi hataları puan kırarken en son bakılan noktadır ve orada büyük puan kırılmaz.

Hoca derste bu soru için kod paylaşmadı. Anlatılan adımlara göre yazılmış bir çözüm "Örnek Sorular" bölümündedir.

5. Böl ve Fethet ile En Küçüğü Bulma

5.1 Problem ve Asıl İstenen

Dördüncü klasik soruda n yarışmacının bir koşudaki bitirme süreleri (saat, dakika, saniye) veriliyor ve altın madalyayı hangi yarışmacının kazandığı soruluyordu. Hoca, bu soruya sınıfın hemen hemen tamamının başka bir çözüm önerdiğini, doğrusunu konuşmanın bu yüzden önemli olduğunu söyledi.

İlk önemli nokta, hangi çözüm yapılırsa yapılsın sorunun altın madalya kazanan yarışmacının bilgisini istemesidir. Çoğu kişi diziyi sıralamış ve sıralanmış bütün koşucuların bilgilerini ekrana yazdırmıştı. Bu istenen bilgi değildir. Sıraladıysanız bile altın madalyayı hangi yarışmacının kazandığını açıkça söylemeniz gerekir.

İkinci nokta, problemin gerçekte ne olduğudur. Altın madalyayı kazanan kişi birinci olmuş, yani en kısa süreyi yapmış kişidir. Bunu bulmak için saat, dakika ve saniye aynı birime çevrilir. Saniye cinsinden en küçük değere sahip koşucu, sırt numarası üzerinden altın madalyanın sahibi olarak belirlenir. Yani bu bir **en küçüğü bulma** problemidir.

Soru (hoca): Bir veri kümesinde en küçüğü bulmak için normalde ne yapıyoruz? **Cevap (bir öğrenci):** Bir tur dönerek bir değişkende en küçüğü tutabiliyoruz. Hoca onayladı. Bu iş kaba kuvvetle (brute force) yapılsaydı, eldeki elemanlara bir turda bakılırdı. Bu aslında sıralamanın ilk turudur, en büyük ya da en küçük elemanı bulmaktır. Bu yüzden kaba kuvvetin karmaşıklığı $O(n)$'dir.

> **Hocanın vurgusu:** Birinci en küçüğü ya da birinci en büyüğü bulmak istediğimiz diziyi hiçbir koşulda sıralamayız. Sıralasak da yalnız bir tur sıralarız. Sınıfın yaklaşık %90'ı quick sort ya da merge sort kullandı; bu çözümlerin ikisi de istenen çözüm değildir.

Hoca ikinci bir uyarı yaptı: Herhangi bir böl ve fethet (divide and conquer) algoritmasıyla soru çözülebilir, ama amaç probleme uygun olmayan hazır bir böl ve fethet algoritmasını alıp uygulamak değildir.

> **Hocanın vurgusu:** Böyle sorularda özyinelemeli bir mantık kurarken "daha önce gördüğüm hazır çözümler mutlaka bir çözümdür" düşüncesinden yola çıkmayın. Problemin gerçekte ne istediğine bakın. Soruda sizi buraya yönlendirecek özel bir ifade de yoktu.

Ayrıca yarışmacı bilgilerinin bir `struct` içinde tutulması bekleniyordu. Hiç `struct` kullanmadan merge sort'u doğrudan bir diziye uygulayanlardan puan kırıldı.

5.2 Böl ve Fethet Çözümü

Hoca soruyu sınıfla tartışırken bir öğrenci kendi çözümünde en uç elemanı "ikili arama (binary search) gibi" bulduğunu söyledi. Hoca çözümü anlatırken bu bakış açısına geri döndü.

Hoca önce derste verdiği bir örneği hatırlattı: a^n hesaplanırken her seferinde $a \cdot a \cdot a \cdots$ yapmak yerine problem $a^{n/2}$ biçiminde aşağıya bölünerek böl ve fethetle de hesaplanabilir. Burada da mantık aynıdır: Dizi ikiye bölünür, her yarının en küçüğü bulunur ve bu ikisi karşılaştırılır.

Hocanın ekranda gösterdiği kodda fonksiyonun adı `max_index` idi. Hoca adın önemli olmadığını, aslında en küçüğün indisinin hesaplandığını söyledi. Fonksiyonun yapısı şöyledir:

- Diziye ek olarak dizinin sol (left) ve sağ (right) sınırları gönderilir. Böylece istenirse dizinin yalnız belli bir bölgesinde de arama yapılabilir. Bütün dizide aramak için ilk çağrı 0 ve $n-1$ ile yapılır.
- `l == r` ise tek elemanlık parçaya inilmiştir ve `return l` ile o indis döndürülür. Bu noktada karşılaştırma yapılmaz.
- Değilse fonksiyon kendini iki kez çağırır: sol parça için `l` ile `(l + r) / 2` arası, sağ parça için `(l + r) / 2 + 1` ile `r` arası. Hoca bunun ikili aramadaki mantığa çok benzediğini söyledi: bir tarafta sol parça, diğer tarafta sağ parça alınır.
- Sol çağrının dönüş değeri `temp1`'e, sağ çağrının dönüş değeri `temp2`'ye atanır.
- `a[temp1] <= a[temp2]` ise `temp1`, değilse `temp2` döndürülür. En büyüğü arasaydık eşitsizliği ters çevirip büyük eşit koymamız gerekirdi.

Böylece fonksiyonlar birbirini çağırarak dizinin en küçük parçalarına kadar iner, her dönüşte o parçanın en küçük elemanını yukarı taşır ve en sonunda bütün dizinin en küçüğünün indisini döndürür.

Aşağıdaki kod, hocanın anlattığı yapının yarışmacı `struct`'ı ile yazılmış hâlidir. Fonksiyon gövdesi hocanın anlattığı adımların aynısıdır; `struct` alanları ve `main` ara adım olarak eklendi. Kod `gcc -Wall -Wextra` ile uyarısız derlendi ve çalıştırıldı.

```c
#include <stdio.h>

typedef struct {
    int no;          /* sırt numarası */
    int h, m, s;     /* bitirme süresi */
    int seconds;     /* saniyeye çevrilmiş süre */
} Runner;

int min_index(Runner a[], int l, int r) {
    if (l == r)
        return l;
    int mid = (l + r) / 2;
    int t1 = min_index(a, l, mid);        /* sol yarının en küçüğü */
    int t2 = min_index(a, mid + 1, r);    /* sağ yarının en küçüğü */
    return (a[t1].seconds <= a[t2].seconds) ? t1 : t2;
}

int main(void) {
    Runner a[100];
    int n, i;
    if (scanf("%d", &n) != 1 || n < 1 || n > 100)
        return 1;
    for (i = 0; i < n; i++) {
        scanf("%d %d %d %d", &a[i].no, &a[i].h, &a[i].m, &a[i].s);
        a[i].seconds = a[i].h * 3600 + a[i].m * 60 + a[i].s;
    }
    i = min_index(a, 0, n - 1);
    printf("Altin madalya: %d numarali yarismaci (%d:%02d:%02d)\n",
           a[i].no, a[i].h, a[i].m, a[i].s);
    return 0;
}
```

Hoca, bunun çok zor olmadığını, daha önce a^n ile ilgili yapılan örnekte bunun daha değişik bir biçiminin yapıldığını söyledi.

5.3 Karşılaştırma Sayısının Rekürans Bağıntısı

Hoca rekürans bağıntısını yazarken temel işlem olarak karşılaştırmayı seçti ve bu yüzden bağıntıya $C(n)$ adını verdi. İkiye bölünerek gidildiği için ne yazılması gerektiğini sınıfa sordu ve kendisi tamamladı: Bir tarafta $C(n/2)$, diğer tarafta yine $C(n/2)$ kadar karşılaştırma yapılır, ayrıca iki sonucu karşılaştırmak için 1 karşılaştırma eklenir. Tek elemanlık parçada (`l == r`) karşılaştırma yapılmadığı için başlangıç koşulu sıfırdır:

$$C(n) = 2C(n/2) + 1 \quad (n > 1), \qquad C(1) = 0$$

$n = 2^k$ alınır ve geriye doğru yerine koyma uygulanır:

$$C(2^k) = 2\,C(2^{k-1}) + 1$$

$$= 2\left[2\,C(2^{k-2}) + 1\right] + 1 = 2^2\,C(2^{k-2}) + 2 + 1$$

$$= 2^2 \left[2\,C(2^{k-3}) + 1\right] + 2 + 1 = 2^3\,C(2^{k-3}) + 2^2 + 2 + 1$$

Genelleştirme:

$$C(2^k) = 2^i\,C(2^{k-i}) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1$$

$i = k$ konursa $C(2^0) = C(1) = 0$ olur ve ilk terim sıfırlanır:

$$C(2^k) = 2^k \cdot 0 + 2^{k-1} + 2^{k-2} + \cdots + 1 = 2^k - 1$$

Son toplam için hoca formülün daha önce verildiğini hatırlattı. 2^k yerine n konunca:

$$C(n) = n - 1$$

Yani bu böl ve fethet yaklaşımı $n - 1$ karşılaştırma yapar.

> **Ders sonrası not:** Fonksiyon python ile 1'den 199'a kadar her n için rastgele dizilerde çalıştırıldı. Her durumda doğru en küçüğü buldu ve tam $n - 1$ karşılaştırma yaptı. Yani sonuç yalnız $n = 2^k$ için değil, her n için geçerlidir. Eşitlikte <= kullanıldığı için birden çok en küçük varsa en soldaki döndürülür.

5.4 Kaba Kuvvet mi, Böl ve Fethet mi?

Sorunun (c) şıkkı şuydu: Bu problemi böl ve fethet yerine kaba kuvvetle çözmek avantaj mıdır, dezavantaj mıdır?

Soru (hoca): Hangisi avantajlıdır? **Cevap (bir öğrenci):** Kaba kuvvet daha avantajlıdır, çünkü böl ve fethet yaparken sürekli fonksiyon çağırıyoruz; karmaşıklık açısından da bir avantajı yok. Hoca bunun doğru cevap olduğunu söyledi.

Karmaşıklık açısından iki yöntem arasında fark yoktur: İkisi de $n - 1$ karşılaştırma yapar. Hoca daha önceki derslerde a^n örneğinde de konuşulduğunu hatırlattı: Böl ve fethet yaklaşımları her zaman avantaj getirmez. Karmaşıklıklar eşitse, fonksiyon çağrısı yükü (function call overhead) yüzünden böl ve fethet kaba kuvvetten daha dezavantajlı olabilir. Burada da böl ve fethet kendini çağırdığı için bu yükü taşır. Beklenen cevap "böl ve fethet kaba kuvvete göre dezavantajlıdır" idi.

> **Ek bilgi (derste anlatılmadı):** Fonksiyon çağrısı yükü; her çağrıda parametrelerin ve dönüş adresinin yığına (call stack) konması, yerel değişkenlere yer açılması ve dönüşte bunların geri alınmasından oluşur. Bu yöntemde toplam $2n - 1$ çağrı yapılır. Ayrıca özyineleme derinliği $\lceil \log_2 n \rceil$ kadar ek yığın belleği kullanır; kaba kuvvet döngüsü ise sabit ek bellekle çalışır.

6. Dönem Projesinin Teknik Konusu: İşbirlikçi Filtreleme ile Kitap Önerisi

Hoca proje dokümanını ekranda paylaşarak projenin teknik içeriğini anlattı. Projede bir kitap öneri sistemi tasarlanacak ve işbirlikçi filtreleme (collaborative filtering) yöntemi kullanılacak. Hoca, bugün birçok önemli kurumun kullanıcılarına öneride bulunan makine öğrenmesine dayalı sistemleri olduğunu, bunların kişiyi platformda daha mutlu edecek seçenekleri sunmaya çalıştığını söyledi. Projenin algoritma analizi dersinde görülen konular dışında bir alandan seçilmesinin sebebi, öğrencilerin farklı konularda da bakış açısı kazanmasıydı. Sistemin üç bölümü vardır.

6.1 Okuyucular Arası Benzerlik: Pearson Katsayısı

Bir okuyucunun diğer okuyuculara benzerliği Pearson katsayısıyla hesaplanır. Bu bir benzerlik skorudur (similarity score) ve -1 ile $+1$ arasında bir değer üretir. Dokümandaki gösterimde:

- P: hem a hem b tarafından okunmuş kitaplar.
- $r_{a,p}$: a'nın p kitabına verdiği puan; $r_{b,p}$ de b'nin verdiği puan.
- $\bar{r}_a$: a'nın okuduğu kitapların puan ortalaması; $\bar{r}_b$ de b'ninki.

$$sim(a,b) = \frac{\sum_{p\in P}(r_{a,p}-\bar{r}_a)(r_{b,p}-\bar{r}_b)}{\sqrt{\sum_{p\in P}(r_{a,p}-\bar{r}_a)^2}\,\sqrt{\sum_{p\in P}(r_{b,p}-\bar{r}_b)^2}}$$

Hoca dokümanda ortalama sembollerinin üstüne çizgi konmadığını belirtti. Hesap için önce her okuyucunun puan ortalaması çıkarılır, sonra iki okuyucunun ortak kitaplara verdiği puanlar kullanılarak a ile b arasındaki skor hesaplanır. Benzerlikte $+1$'e yakın, yüksek olan değer tercih edilir; belirleyici olan budur.

6.2 En Benzer k Okuyucu

Sisteme bir k değeri girilir ve okuyucuya en çok benzeyen k okuyucu bulunur. Hoca burada vizedeki dördüncü soruyla bağlantı kurdu: En benzer k kişiyi bulmak için bütün benzerlikler sıralanmaz, daha verimli bir çözüm kullanılmalıdır. Bu nokta, sınavdaki sıkıntı tekrar yaşanmasın diye dokümanda özellikle vurgulanmıştı.

Soru: İkinci şıkta "daha verimli bir çözüm planlayın" deniyor; bunun için çok zor bir şey mi düşünmek gerekiyor? **Cevap:** Hayır, cevap çok basit. Sınavdaki gibi üzerine çok düşünmeyi gerektiren bir şey değil, sadece dikkat çekildi. Birinci sınıfı bitiren birinin bileceği bir şey.

> **Ek bilgi (derste anlatılmadı):** n okuyucunun benzerliklerini tamamen sıralamak $O(n \log n)$ sürer. k küçükse en büyüğü bulma işlemini k kez tekrarlamak (selection sort'un ilk k turu) $O(kn)$ sürer. k boyutlu bir min-heap ile $O(n \log k)$'de, hızlı seçim (quickselect) ile ortalama $O(n)$'de de bulunabilir.

6.3 Yeni Kitap Önerme

Okuyucuya kitap önermek için bulunan en benzer k kişi kullanılır. Önce bu k kişi bir yerde saklanır. Sonra okuyucunun okumadığı her kitap için, bu k kişinin bilgisi kullanılarak bir tahmini puan hesaplanır. Bütün okunmamış kitapların skorları hesaplandıktan sonra en yüksek skorlu kitap okuyucuya önerilir. Hoca projenin bundan ibaret olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Kullanıcı tabanlı işbirlikçi filtrelemede tahmin genelde $pred(a,p) = \bar{r}_a + \dfrac{\sum_{b\in N} sim(a,b)\,(r_{b,p}-\bar{r}_b)}{\sum_{b\in N} sim(a,b)}$ biçimindedir; N, en benzer k kişidir. Dokümandaki formül büyük olasılıkla bu biçimdedir. Benzerlikler negatif olabildiği için literatürde payda çoğu zaman $\sum_{b\in N}|sim(a,b)|$ olarak alınır; aksi hâlde payda sıfıra yaklaşabilir ya da işaret değiştirebilir.

6.4 Projeyle İlgili Sorular

Soru: Sistemin doğru öneri yapıp yapmadığını nasıl test edeceğiz? Belki yanlış hesaplıyoruzdur. **Cevap:** Hesap çok açık. Formüller uygulandığında çıkan skorlara bakılabilir ve bir ya da iki okuyucu için bu skorlar elle kontrol edilebilir. Doğruluğu belirlemek için örneğe ihtiyaç duyulursa hoca bunu diğer grubun hocasıyla konuşup örnek verilip verilmeyeceğini bildirecekti.

Soru: Veriler XLS dosyasında; dosyayı doğrudan okumak şart mı, verileri koda alabilir miyiz? **Cevap:** Olur. Amaç dosya okuma kodu yazmak değil. Veriler doğrudan koda alınabilir ya da dosya CSV'ye çevrilip okunabilir; CSV okumak kolaydır. Önemli olan ödevi kendinizin yapması.

Örnek Sorular

Bu derste kare yoktur. Sorular, hocanın derste okuyup çözdüğü vize sorularından ve derste sözlü sorulan sorulardan alındı.

1. (derste sözlü, 00:03:41) Sayacı her adımda iki katına çıkan bir döngü içindeki toplama işlemi kaç kez yapılır?

- Çözüm: $\log n$ kez. $n = 8$ için 3, $n = 16$ için 4 kez.

2. (derste sözlü, 00:04:15) Linear probing kullanan bir hash tablosunda boş gözler "empty" ya da "deleted" bilgisini taşır. (i) i. adres deleted, $i + 1$. adres empty ise i. adres empty işaretlenebilir. (ii) i. adres deleted, $i + 1$. adreste doğru adrese yerleşmiş bir eleman varsa i. adres empty işaretlenebilir. Doğru mu, yanlış mı?

- Çözüm: (i) doğru, arama zaten $i + 1$'de durur. (ii) yanlış, i empty yapılırsa i'nin ötesine yerleşmiş elemanların araması yarıda kesilir.

3. (derste sözlü, 00:05:22) $2n^3 - 7n + 1$ için $O(n^3)$, $\Theta(n^3)$, $O(n^4)$, $\Omega(n)$ gösterimlerinden hangisi kullanılamaz?

- Çözüm: Hepsi kullanılabilir.

4. (derste sözlü, 00:06:10) Sayılar $h(x) = x$ mod 10 ile linear probing kullanılarak yerleştiriliyor. 6 adım sonra tablo adres 2-7'de sırasıyla 42, 23, 34, 52, 46, 33 içeriyor. Sayılar hangi sırayla yerleştirilmiştir?

- Çözüm: 46, 34, 42, 23, 52, 33. 52, 2-3-4 dolu olduğu için 5'e; 33, 3-4-5-6 dolu olduğu için 7'ye yerleşir. Python ile yerleştirme yapılarak tablo doğrulandı.

5. (derste sözlü, 00:07:33) Yer karmaşıklığı, zaman karmaşıklığı, çalışma süresi, büyüme derecesi, girdi sayısı, kod uzunluğu; bunlardan kaç tanesi iki algoritmanın verimlilik yönünden karşılaştırılmasında ölçüt olabilir?

- Çözüm: 4. Girdi sayısı ve kod uzunluğu ölçüt değildir.

6. (derste sözlü, 00:08:47) n elemanlı ikili (arama) ağacına eleman eklemenin en kötü durum karmaşıklığı nedir?

- Çözüm: $O(n)$. Ağaç bir tarafa yığılabilir.

7. (derste sözlü, 00:09:10) Bir dizinin beş sıralama algoritmasıyla sıralanırken bir ara adımdaki hâlleri sütunlar hâlinde veriliyor. Sütunları seçmeli, hızlı, birleştirmeli, eklemeli ve yığın sıralamasıyla eşleştirin.

- Çözüm: Sıra selection, quick, merge, insertion, heap. İki yarısı ayrı ayrı sıralı olan merge, başı sıralı ve kalanı ilk sırasını koruyan insertion, başı en küçüklerden oluşup kalanın sırası bozulmuş olan selection sort'tur.

8. (derste sözlü, 00:15:09) Her satıra yalnız O, Θ veya Ω yazın: n^2 – n^3; 1 – $2 + \sin n$; $n \log n$ – $n^{150/100}$; $\log(\sqrt{10}\, n)$ – $\log(n^3)$; $n!$ – $(n + 1)!$.

- Çözüm: O; Θ; O; Θ; O. Ayrıca bir satırın cevabı Ω'dır.

9. (derste sözlü, 00:16:43) $2 + \sin n$ satırına ne yazılmalı?

- Çözüm: Θ. $1 \leq 2 + \sin n \leq 3$, yani ifade sabittir ve n ile büyümez.

10. (derste sözlü, 00:20:59) $x_n = x_{n-1} + 5$ $(n > 1)$, $x_1 = 0$ bağıntısını adımları göstererek çözün.

- Çözüm: $x_n = x_{n-i} + 5i$, $i = n - 1$ ile $x_n = 5n - 5$. Python ile $n = 1, \ldots, 19$ için doğrulandı.

11. (derste sözlü, 00:23:19) $T(n) = 4T(n/2) + n^2$, $T(1) = 1$ bağıntısını çözün.

- Çözüm: $a = 4$, $b = 2$, $d = 2$, $a = b^d$. $T(n) \in \Theta(n^2 \log n)$.

12. (derste sözlü, 00:24:04) Master teoreminde $a = b^d$ durumunda sonuç nedir?

  Çözüm (derste verilmedi): $T(n) \in \Theta(n^d \log n)$. Hoca formülü hatırlatmadan doğrudan sonucu yazdı.

13. (derste sözlü, 00:28:11) İki ayrı dizide verilen iki kümenin kesişimini hash yöntemiyle bulan standart C programını yazın. Çakışmayı linear probing ile çözün.

- Derste verilen çözüm adımları: Asal boyutlu tablo; birinci diziyi kendi yazdığınız hash fonksiyonuyla yerleştirme ve tablonun dolu olup olmadığını kontrol etme; ikinci dizinin her elemanını boş göz görene kadar eşitlik kontrolüyle arama; döngüden çıkış sebebini kontrol etme.

  Çözüm (derste verilmedi): Aşağıdaki kod, derste sayılan adımlara göre yazıldı. Elemanların negatif olmayan tam sayılar olduğu, her kümede elemanların tekrar etmediği ve dizilerin en fazla 100 eleman içerdiği varsayıldı. Kod `gcc -Wall -Wextra` ile uyarısız derlendi; aynı adrese düşen elemanlarla denendi ve sonuç python küme kesişimiyle aynı çıktı.

```c
#include <stdio.h>

#define M 101          /* tablo boyutu: asal sayı */
#define EMPTY -1

int table[M];

int hash(int key) {
    return key % M;
}

/* linear probing ile yerleştirme; tablo doluysa 0 döner */
int insert(int key) {
    int i = 0, adr = hash(key);
    while (table[adr] != EMPTY && table[adr] != key && i < M) {
        adr = (adr + 1) % M;
        i++;
    }
    if (i == M)
        return 0;                  /* tablo dolu */
    table[adr] = key;
    return 1;
}

/* boş göz görene kadar ya da bütün tablo dolaşılana kadar ara */
int search(int key) {
    int i = 0, adr = hash(key);
    while (table[adr] != EMPTY && table[adr] != key && i < M) {
        adr = (adr + 1) % M;
        i++;
    }
    return (i < M && table[adr] == key);   /* çıkış sebebini kontrol et */
}

int main(void) {
    int a[100], b[100], n, m, i;
    for (i = 0; i < M; i++)
        table[i] = EMPTY;
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &m);
    for (i = 0; i < m; i++) scanf("%d", &b[i]);
    for (i = 0; i < n; i++)
        if (!insert(a[i]))
            printf("Tablo dolu, %d yerlestirilemedi\n", a[i]);
    printf("Kesisim:");
    for (i = 0; i < m; i++)
```

```
        if (search(b[i]))
            printf(" %d", b[i]);
    printf("\n");
    return 0;
}
```

14. (derste sözlü, 00:32:52) n yarışmacının bitirme süreleri (saat, dakika, saniye) veriliyor. (a) Altın madalyayı kazananı böl ve fethet yaklaşımıyla bulan C programını yazın. (b) Karşılaştırma sayısının rekürans bağıntısını yazıp çözün. (c) Böl ve fethet yerine kaba kuvvet kullanmak avantaj mıdır?

- Çözüm (a): Süreler saniyeye çevrilir, yarışmacılar `struct` dizisinde tutulur ve diziyi ikiye bölüp iki yarının en küçüğünü karşılaştıran `min_index` fonksiyonu çağrılır (Bölüm 5.2). Dizi sıralanmaz.
- Çözüm (b): $C(n) = 2C(n/2) + 1$, $C(1) = 0$; $C(n) = n - 1$ (Bölüm 5.3). Python ile her $n \leq 199$ için doğrulandı.
- Çözüm (c): Karmaşıklık aynıdır ($n-1$ karşılaştırma). Böl ve fethet fonksiyon çağrısı yükü yüzünden dezavantajlıdır.

15. (derste sözlü, 00:34:37) Bir veri kümesinde en küçüğü bulmak için normalde ne yaparız?

- Çözüm: Diziyi bir kez dolaşıp en küçüğü bir değişkende tutarız. Bu kaba kuvvet çözümü $O(n)$'dir ve $n - 1$ karşılaştırma yapar.

Dersin Toparlaması

Bu son derste yeni konu anlatılmadı, ama vize soruları dönemin ana konularını toparladı: döngü analizi, asimptotik gösterimlerin anlamı ve fonksiyonların büyüme hızlarının karşılaştırılması, geriye doğru yerine koyma ve master teoremiyle rekürans çözümü, linear probing ile hash tablosunda yerleştirme, arama ve silme, ikili ağacın en kötü durumu ve sıralama algoritmalarının ara adımlarındaki izlerinden tanınması.

Dersin ana mesajı dördüncü soruda verildi: Bir problemi çözerken önce problemin gerçekte ne istediğine bakılmalıdır. En küçüğü ya da en büyüğü bulmak için dizi sıralanmaz. Böl ve fethetle çözülecekse hazır bir sıralama algoritması uygulanmaz, problem doğrudan ikiye bölünüp iki parçanın sonuçları birleştirilir. Bu çözüm de kaba kuvvetle aynı sayıda ($n - 1$) karşılaştırma yapar, üstüne fonksiyon çağrısı yükü taşıdığı için daha avantajlı değildir. Aynı ilke dönem projesinde de geçerlidir: İşbirlikçi filtrelemede en benzer k okuyucu bulunurken bütün benzerlikler sıralanmamalı, daha verimli bir seçim yöntemi kullanılmalıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.